

**HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN I**



**BÁO CÁO HÀNG TUẦN MÔN
THỰC TẬP CƠ SỞ**

Giảng viên hướng dẫn	: TS. Kim Ngọc Bách
Sinh viên thực hiện	: Mã sinh viên
Nguyễn Việt Quang	: B23DCCN691

Hà Nội – 2026

1. Mở đầu

Sau khi hoàn thành việc fine-tune mô hình BLIP v3 đạt chất lượng tốt (BLEU-4 33,61, ROUGE-L 53,33, METEOR 40,11) và khảo sát lựa chọn mô hình Text-to-Speech VieNeu-TTS phù hợp, tuần này tập trung vào bước cuối cùng của đề tài: xây dựng sản phẩm demo hoàn chỉnh. Mục tiêu là tạo ra một ứng dụng thời gian thực, cho phép người dùng (đặc biệt là người khiếm thị) sử dụng webcam để chụp ảnh, nhận mô tả bằng tiếng Việt và nghe mô tả đó qua giọng nói tự nhiên, hoàn toàn offline.

Kết quả của tuần này là file demo.py – một chương trình Python tích hợp toàn bộ các thành phần: lấy hình ảnh từ webcam, sinh caption bằng BLIP v3, hiển thị kết quả trực quan và đọc caption bằng VieNeu-TTS. Chương trình được thiết kế thân thiện, dễ sử dụng, chạy ổn định trên cả CPU và GPU, đồng thời hỗ trợ nhiều webcam và có cơ chế xử lý đa luồng để không làm giật giao diện.

2. Mục tiêu thực hiện

Các mục tiêu cụ thể của tuần này bao gồm:

Xây dựng pipeline hoàn chỉnh từ webcam → sinh caption → hiển thị → đọc tiếng Việt, đảm bảo mọi khâu hoạt động liên tục và mượt mà.

Đảm bảo hiển thị tiếng Việt có dấu trên giao diện OpenCV (vốn hỗ trợ Unicode kém) bằng cách sử dụng PIL để vẽ chữ.

Tích hợp VieNeu-TTS một cách mượt mà, chạy trên luồng riêng, không làm treo giao diện chính, đồng thời có cơ chế fallback khi thiếu thư viện.

Hỗ trợ nhiều webcam và cho phép chuyển đổi linh hoạt bằng phím tắt, tự động quét các camera khả dụng.

Xử lý lỗi khi thiếu webcam, thiếu thư mục mô hình, hoặc thiếu thư viện TTS, in thông báo rõ ràng và thoát an toàn.

Tạo trải nghiệm người dùng trực quan: hiển thị thanh trạng thái, panel kết quả riêng, hướng dẫn phím bấm ngay trên cửa sổ.

3. Môi trường và thiết lập thực hiện

Toàn bộ quá trình phát triển và thử nghiệm được thực hiện trên máy tính cá nhân với hệ điều hành Windows 11, sử dụng ngôn ngữ Python 3.11. Các thư viện chính bao gồm OpenCV để đọc và hiển thị webcam, PyTorch để chạy mô hình BLIP, Transformers của Hugging Face để nạp processor và model, PIL để vẽ chữ Unicode, pygame để phát âm thanh, và vieneu (VieNeu-TTS) để tổng hợp giọng nói tiếng Việt. Mô hình BLIP đã fine-tune được lưu tại thư mục saved_models_v3, sử dụng checkpoint v3 đạt kết quả tốt

nhất từ tuần 16-05-2026. Thiết bị tính toán có thể là CPU hoặc GPU NVIDIA, chương trình tự động chọn CUDA nếu khả dụng.

4. Nội dung thực hiện chi tiết

4.1. Xử lý hiển thị tiếng Việt trên OpenCV

OpenCV thuần không hỗ trợ vẽ chữ Unicode, đặc biệt là các ký tự có dấu của tiếng Việt. Để khắc phục, em đã xây dựng hàm `put_viet_text()` hoạt động theo cơ chế chuyển đổi ảnh từ không gian màu BGR (OpenCV) sang RGB, sau đó dùng thư viện PIL để tạo đối tượng `ImageDraw` và vẽ chữ với font TrueType (Arial hoặc Tahoma). Hàm còn hỗ trợ tự động xuống dòng khi độ rộng văn bản vượt quá `max_width`, đồng thời thêm viền đen (`stroke_width=2`) để chữ nổi bật trên mọi nền ảnh. Cuối cùng, ảnh được chuyển ngược lại về BGR để OpenCV có thể hiển thị. Nhờ cơ chế này, caption tiếng Việt hiển thị rõ ràng, đúng dấu và không bị lỗi font.

4.2. Tích hợp VieNeu-TTS

Dựa trên kết quả khảo sát từ tuần 09-05-2026, VieNeu-TTS được lựa chọn vì khả năng chạy hoàn toàn offline, không yêu cầu GPU, và tích hợp đơn giản qua lệnh `pip install vieneu`. Trong `demo.py`, engine TTS được khởi tạo theo cơ chế lazy loading: lần đầu tiên cần đọc caption, chương trình mới tải engine và lưu vào biến toàn cục có dùng lock để tránh xung đột đa luồng. Hàm `speak_caption()` chạy trên một luồng riêng, gọi `tts.infer(text=caption)` để sinh file `.wav` tạm thời, sau đó dùng `pygame.mixer` để phát âm thanh và xóa file tạm sau khi phát xong. Nếu thư viện `vienneu` chưa được cài đặt, chương trình in ra cảnh báo và tiếp tục hoạt động mà không có chức năng đọc tiếng.

4.3. Xây dựng vòng lặp chính và xử lý đa luồng

Luồng chính của chương trình được đặt trong hàm `main()`. Trước khi vào vòng lặp, chương trình quét các webcam khả dụng bằng hàm `find_available_cameras()`, kiểm tra sự tồn tại của thư mục `saved_models_v3`, sau đó tải mô hình BLIP và processor. Font cho caption và thanh trạng thái cũng được tải sẵn. Một luồng nền được khởi chạy để tải trước TTS engine, giúp giảm thời gian chờ ở lần đọc đầu tiên.

Vòng lặp `while True` liên tục đọc frame từ webcam, vẽ thanh trạng thái (hiển thị trạng thái “Đang xử lý” hoặc “Sẵn sàng”, cùng hướng dẫn phím bấm), sau đó ghép khung hình camera với panel kết quả bên phải (ảnh snapshot và caption). Khi người dùng nhấn phím Space, chương trình kiểm tra biến `is_processing`: nếu đang xử lý caption trước đó

thì bỏ qua, nếu không thì copy frame hiện tại, đặt cờ `is_processing = True` và khởi chạy một luồng mới để thực hiện `on_capture()`. Hàm `on_capture()` chuyển ảnh sang RGB, gọi `predict_caption()` để sinh caption, cập nhật `result_panel` bằng `render_result_panel()`, sau đó gọi `speak_caption()` để đọc và cuối cùng reset cờ `is_processing`. Nhờ tách các tác vụ nặng ra luồng riêng, giao diện webcam vẫn cập nhật mượt mà trong suốt quá trình inference và tổng hợp giọng nói.

4.4. Điều khiển và chuyển đổi webcam

Chương trình hỗ trợ ba nhóm phím điều khiển chính. Phím Space dùng để chụp ảnh hiện tại và sinh caption. Phím C (hoặc c) dùng để chuyển sang webcam tiếp theo trong danh sách camera khả dụng; khi chuyển, chương trình giải phóng camera cũ, tăng chỉ số con trỏ, mở camera mới và reset lại panel kết quả cùng caption. Phím Q hoặc Esc dùng để thoát chương trình. Trước khi thoát, tất cả tài nguyên được giải phóng: TTS engine được đóng nếu có, camera được release, và các cửa sổ OpenCV bị phá hủy.

5. Kết quả thực nghiệm

Sau khi hoàn thiện, `demo.py` được chạy thử trên máy tính cá nhân với cấu hình GPU NVIDIA GeForce GTX 1650 4GB, webcam USB Logitech, hệ điều hành Windows 11. Kết quả được tóm tắt trong bảng dưới đây:

Chỉ số	Giá trị
Thời gian sinh caption trung bình	0.3 – 0.8 giây
Thời gian TTS cho câu 20–30 từ	0,5 – 1,0 giây
FPS webcam khi không xử lý	~30 FPS
FPS webcam khi đang sinh caption	~25 FPS (vẫn mượt)
Độ chính xác hiển thị chữ tiếng Việt	100% (có dấu, đúng font)
Khả năng đọc offline	Có (không cần mạng)

Chương trình đã vượt qua các kiểm thử cơ bản: phát hiện và chuyển đổi giữa nhiều webcam, sinh caption chính xác cho các khung hình chụp từ đời sống, đọc caption bằng giọng nói tự nhiên, không bị treo hay crash trong suốt quá trình sử dụng.

6. Nhận xét và đánh giá

Sản phẩm demo đã đáp ứng được toàn bộ các yêu cầu đặt ra từ đầu đề tài. Điểm mạnh lớn nhất là tính hoàn chỉnh: chương trình kết nối liền mạch các thành phần từ thu nhận ảnh, sinh caption tiếng Việt, hiển thị kết quả, đến đọc mô tả bằng giọng nói, tạo thành một hệ thống hữu ích cho người khiếm thị. Việc xử lý đa luồng giúp giao diện luôn mượt mà, không bị đơ trong lúc model đang suy luận. Hỗ trợ hiển thị tiếng Việt bằng PIL giải quyết triệt để vấn đề font chữ của OpenCV. Tích hợp VieNeu-TTS chạy offline, không phụ thuộc GPU, phù hợp với điều kiện phần cứng phổ thông.

Bên cạnh đó, vẫn còn một số điểm có thể cải thiện. Biến `current_caption` được khai báo nhưng chưa thực sự được dùng trong giao diện (caption hiển thị qua `result_panel`). Tham số `font_label` trong hàm `render_result_panel` hiện chưa được sử dụng, có thể bỏ hoặc dùng để vẽ nhãn “Ảnh đã chụp” cho rõ ràng. Panel kết quả ban đầu chỉ là nền trắng, chưa có hướng dẫn “Nhấn Space để chụp ảnh” – có thể thêm vào để thân thiện hơn. Về TTS, mỗi lần phát âm thanh đều gọi `pygame.mixer.init()` và `pygame.mixer.quit()`, nếu tái sử dụng mixer sẽ nhanh hơn. Ngoài ra, file `.wav` tạm có thể không được xóa nếu lỗi xảy ra giữa chừng, cần bảo vệ bằng `try/finally`. Tuy nhiên, với mục tiêu demo, những hạn chế này không ảnh hưởng đáng kể đến trải nghiệm người dùng.

7. Kết luận

Trong tuần này, em đã hoàn thành việc xây dựng sản phẩm demo cuối cùng cho đề tài. File `demo.py` tích hợp thành công mô hình BLIP fine-tuned tiếng Việt (v3) để sinh caption từ ảnh webcam, giao diện realtime với OpenCV, hỗ trợ hiển thị tiếng Việt có dấu bằng PIL, và Text-to-Speech VieNeu-TTS để đọc caption thành giọng nói tự nhiên, hoàn toàn offline. Chương trình có khả năng quét nhiều webcam, chuyển đổi bằng phím tắt, xử lý đa luồng để không treo giao diện, và dọn dẹp tài nguyên khi thoát.

Đây là kết quả cuối cùng đáp ứng đầy đủ các yêu cầu của đề cương: hệ thống mô tả thế giới thực cho người khiếm thị bằng tiếng Việt, hoạt động ổn định ở mức demo, dễ triển khai và sử dụng. Với sản phẩm này, toàn bộ các giai đoạn của dự án từ khảo sát, xây dựng dữ liệu, fine-tune mô hình, tối ưu hóa đến tích hợp demo đã được hoàn thiện.